EXPERIMENT 5

Student Name: Ayush Kohli **UID:** 23BCS11238

Branch: BE-CSE **Section/Group:** KRG_3B

Semester: 6th **Date of Performance:** 25/02/2026

Subject Name: System Design **Subject Code:** 23CSH-314

Aim

To design a scalable, real-time messaging application similar to modern communication platforms that supports one-to-one and group messaging while ensuring high availability, low latency, and reliable message delivery.

Objectives

- To understand the architectural components of a large-scale real-time messaging system.
- To identify and define functional requirements such as authentication, messaging, and media sharing.
- To determine non-functional requirements including availability, reliability, scalability, and latency constraints.
- To analyze CAP theorem trade-offs applicable to distributed messaging systems.
- To design RESTful and WebSocket APIs to support real-time communication.

Methodology / Procedure

1. Conducted a study of widely used messaging platforms to understand system design principles and architectural patterns.
2. Identified primary system entities including Users, Groups, and Messages.
3. Examined real-time communication mechanisms using WebSocket protocol.
4. Gathered and categorized functional and non-functional requirements.
5. Designed API endpoints for authentication and messaging operations.
6. Evaluated storage strategies and scalability mechanisms for handling large-scale traffic.

7. Analyzed trade-offs between availability and consistency under distributed system constraints.

Functional Requirements

1. Users must be able to register and authenticate securely.
2. The system must support direct (one-to-one) messaging.
3. The system must enable group-based communication.
4. Both text and media messages (images, videos, documents) must be supported.
5. Complete message history should be maintained for retrieval.
6. Delivery and read receipts must be available to users.
7. Messages should be delivered in real-time with minimal delay.

Core System Entities

- **Users** – Registered participants of the system.
- **Groups** – Collections of users for group-based messaging.
- **Messages** – Communication units exchanged between users or within groups.

API Design

User Authentication APIs

- **User Registration**
  POST /user/register
- **User Login**
  POST /user/login

One-to-One Messaging APIs

- **Send Message (Real-Time via WebSocket)**
  WS /messages/send
- **Retrieve Chat List (Pagination Supported)**
  GET /chat/{user_id}

- **Fetch Messages Between Two Users**
  GET /messages/{user_id}/{receiver_id}

Group Messaging APIs

- **Create Group**
  POST /groups/create
- **Send Group Message (WebSocket)**
  WS /messages/send
- **Retrieve Group Chat Messages (Pagination Supported)**
  GET /groups/{group_id}
- **Add Group Member**
  POST /groups/{group_id}/add
- **Remove Group Member**
  DELETE /groups/{group_id}/remove

Non-Functional Requirements

1. The system must handle massive traffic volumes, potentially processing millions to billions of messages daily, with storage requirements of approximately 100 TB or more.
2. High availability should be prioritized, especially in distributed environments.
3. The system should adopt an eventual consistency model to balance scalability and performance.
4. End-to-end message latency should be maintained within 200–300 milliseconds.
5. Message delivery must be reliable, ensuring no data loss.
6. The architecture must support horizontal scaling across distributed servers.

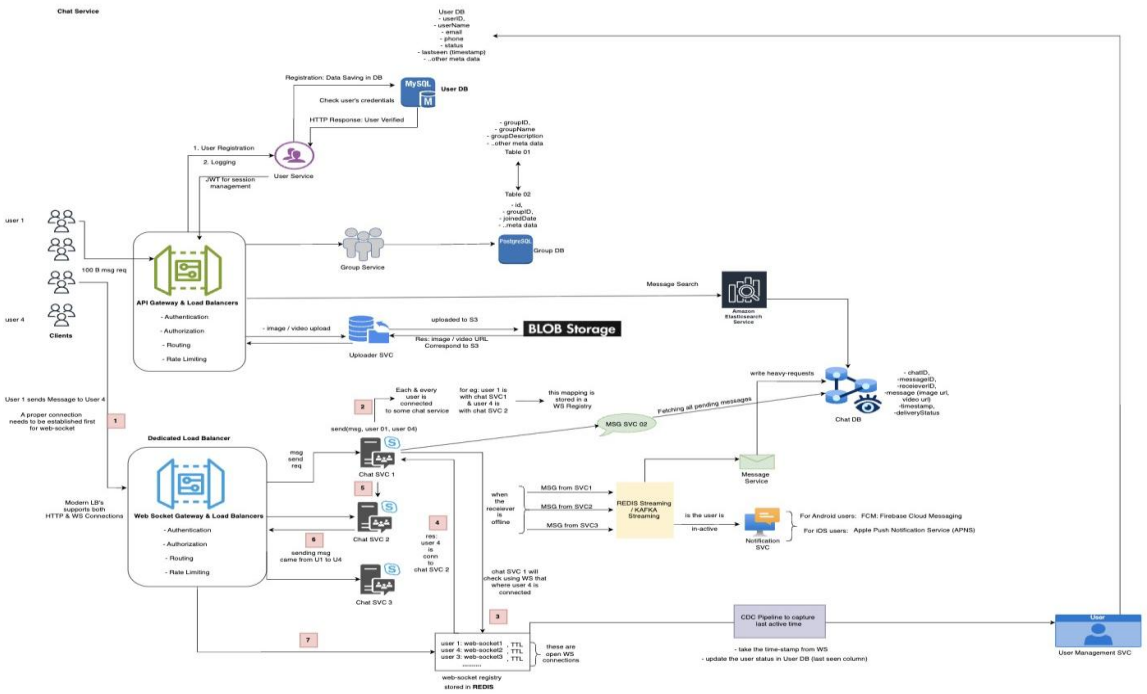
High-Level Design (HLD)

The system architecture consists of the following components:

- **Client Applications** (Mobile and Web)
- **API Gateway** for routing and request handling

- **Authentication Service** for user verification
- **Messaging Service** (WebSocket Servers) for real-time communication
- **Group Management Service**
- **Media Service** for file handling
- **Message Queue** for asynchronous processing
- **Distributed Databases** for persistent storage
- **Cache Layer (e.g., Redis)** for fast data access
- **Object Storage** for media content

Messages are transmitted through WebSocket servers to ensure real-time delivery and are simultaneously persisted in distributed databases to guarantee durability. A message queue ensures reliable asynchronous processing, while caching improves read performance and reduces database load.



Learning Outcomes

- Successfully designed the architecture of a scalable real-time messaging system.
- Systematically identified and classified both functional and non-functional requirements.
- Developed RESTful and WebSocket-based APIs to support real-time communication.
- Gained an in-depth understanding of high availability, eventual consistency, and CAP theorem trade-offs in distributed messaging systems.